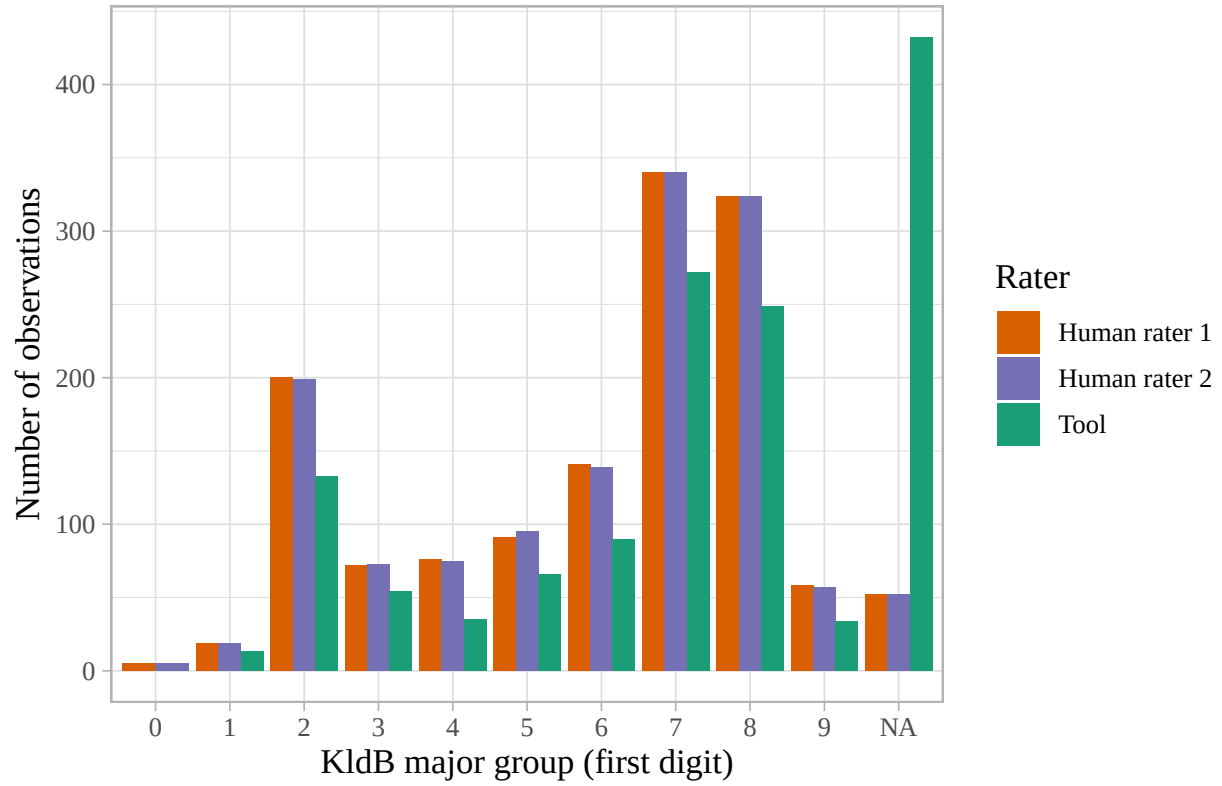
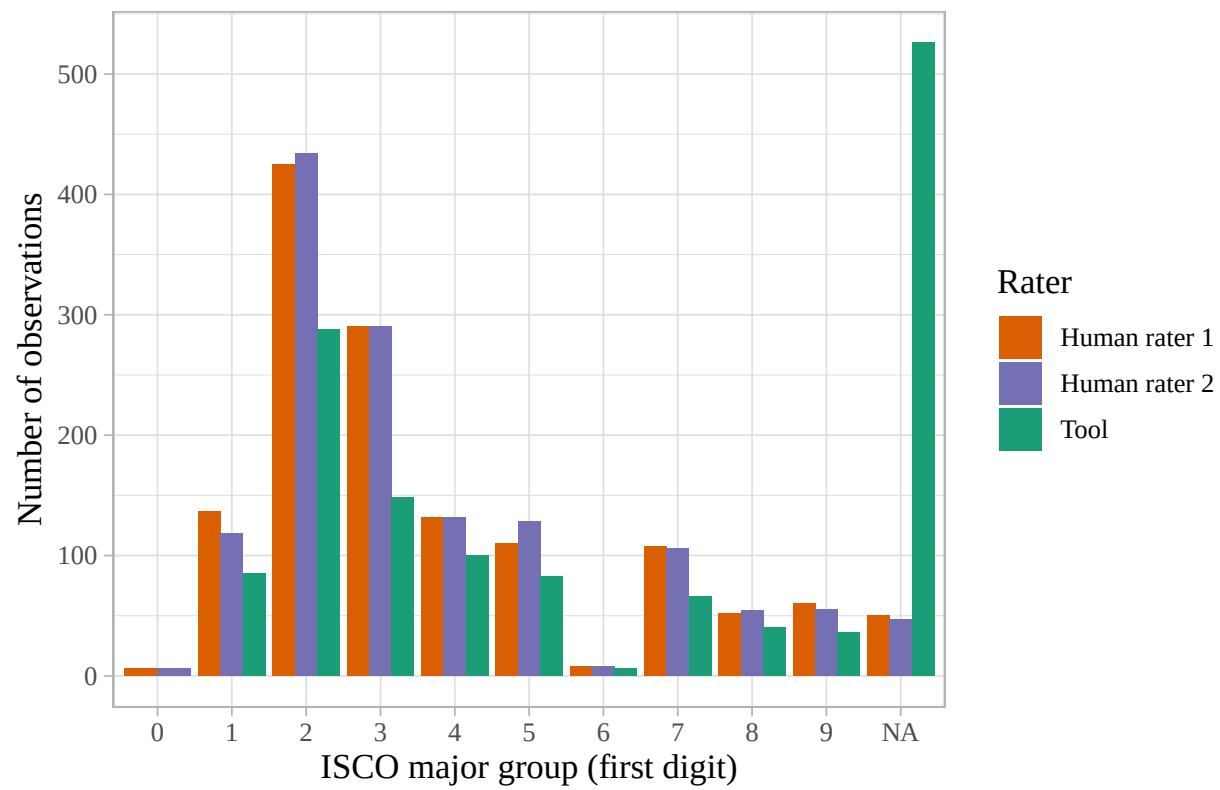


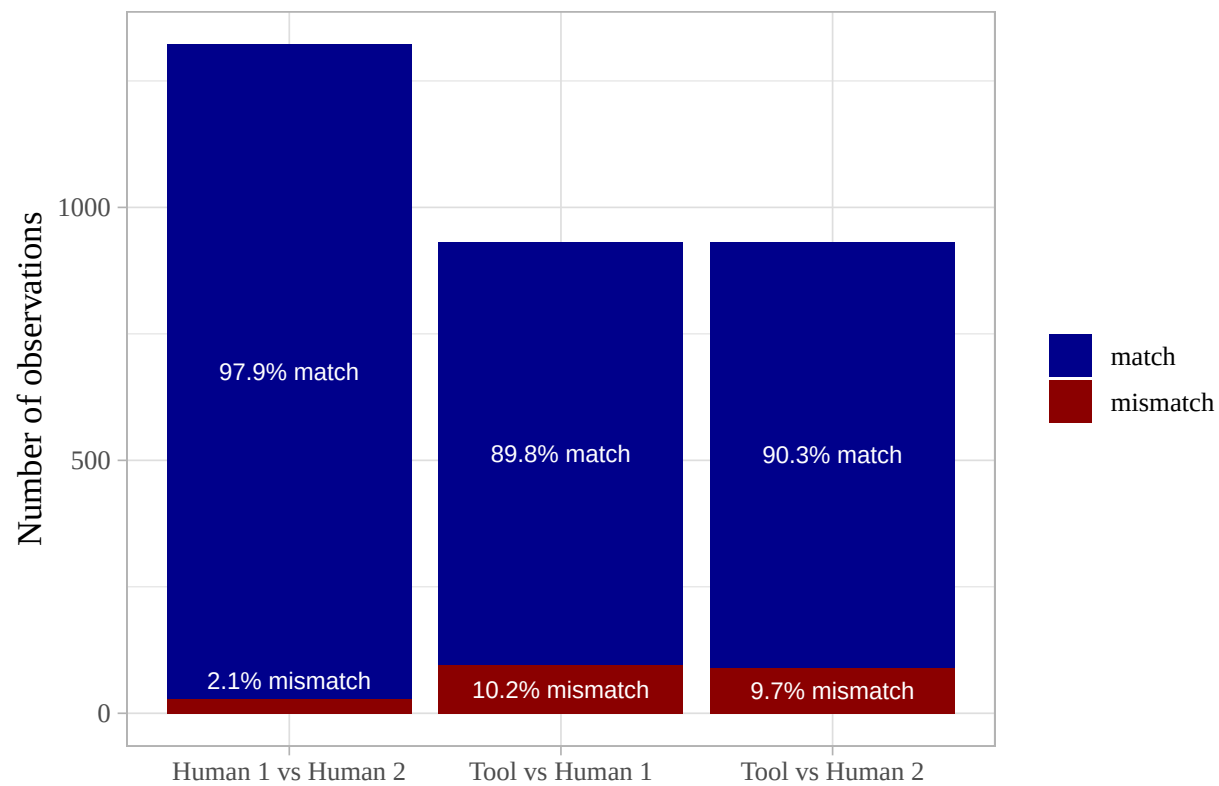
KldB Alpha Beta Parameter Comparison

```
## [1] "C:/Users/jeroe/Uni_C/noisy_measurement/src/data_preparation.R"
```

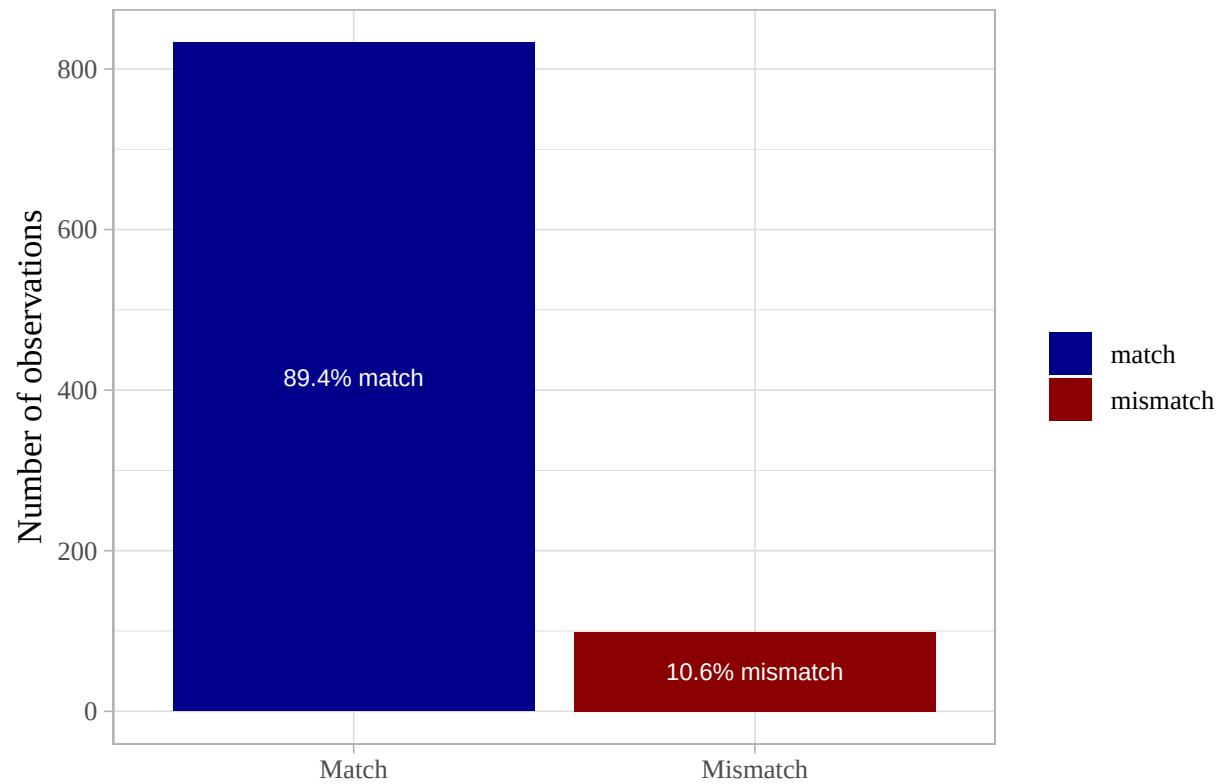




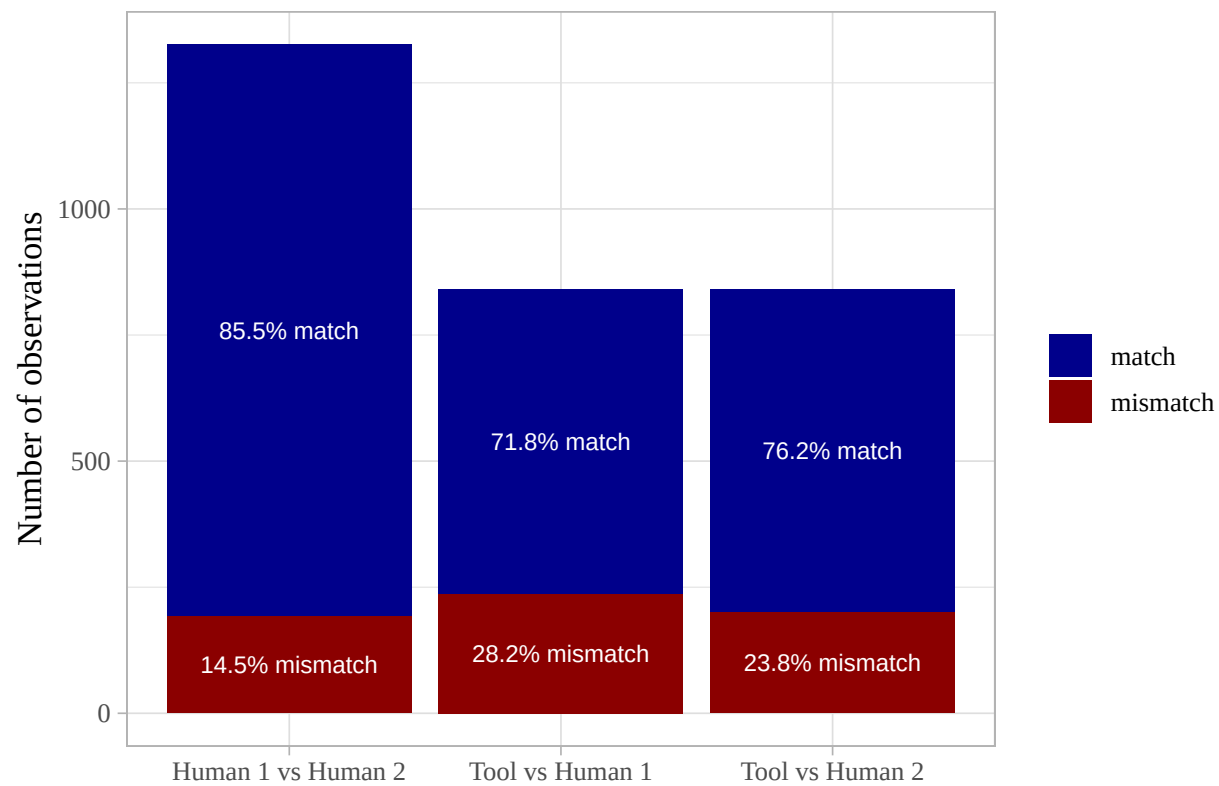
KldB: Pairwise agreement on the first digit



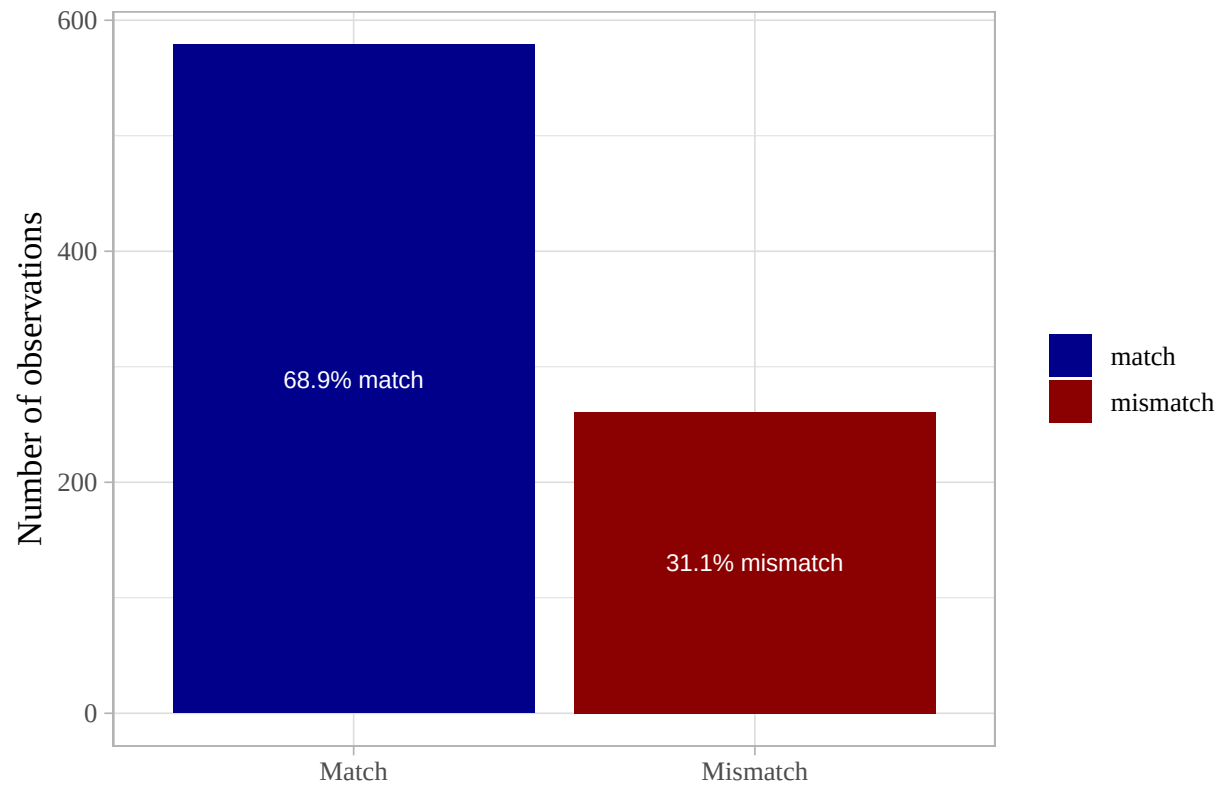
KldB: Agreement across all three sources on the first digit

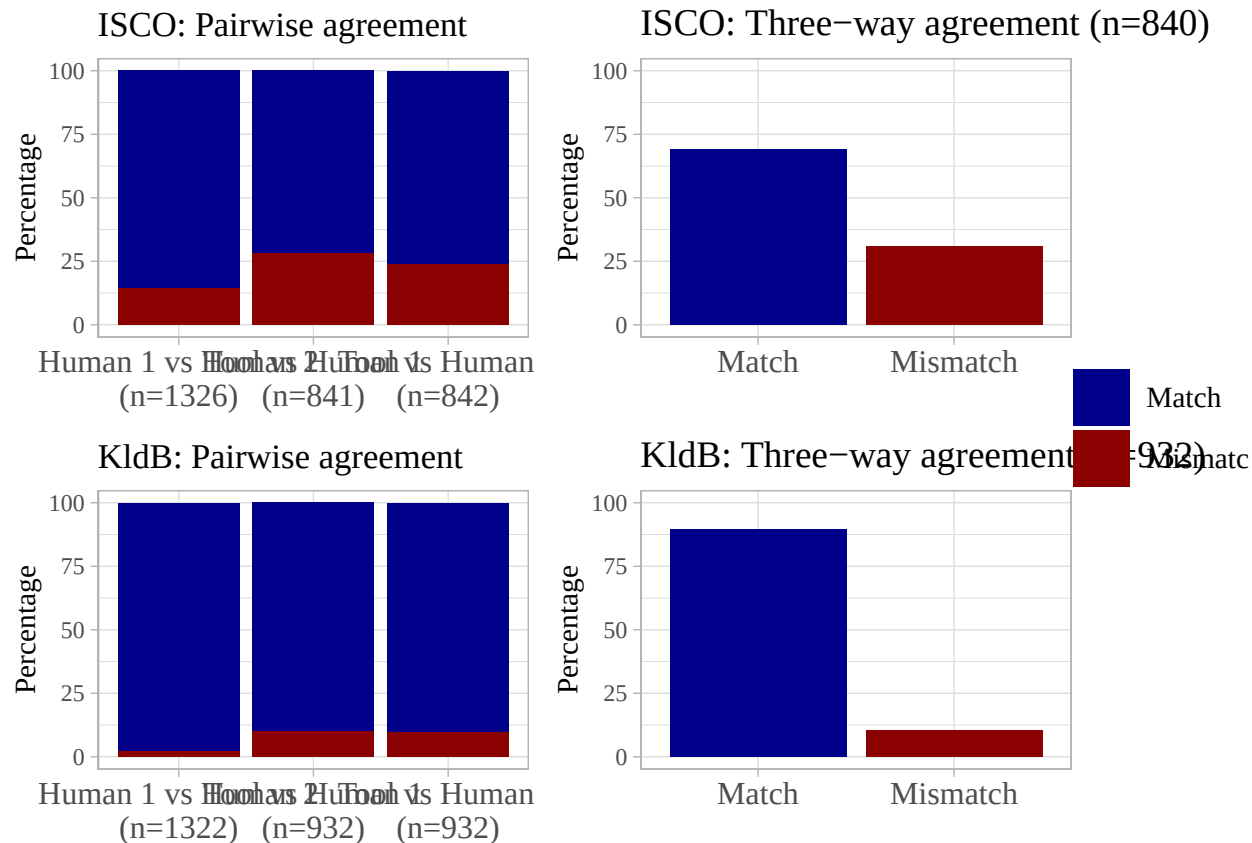


ISCO: Pairwise agreement on the first digit



ISCO: Agreement across all three sources on the first digit





```
# Packages
library(data.table)
library(dplyr)
library(tidyr)
library(cmdstanr)

# keep only relevant columns
kldb_data <- data_basis %>%
  select(item_id, kldb_tool_1st, res1_kldb_1st, res2_kldb_1st)

#Rename columns
setnames(
  data_basis,
  old = c("kldb_tool_1st", "res1_kldb_1st", "res2_kldb_1st"),
  new = c("kldb_tool", "kldb_human1", "kldb_human2")
)

head(data_basis)
```

```
##      kldb_tool_vorlaufig kldb_tool_mit_anfniveau res1_kldb res2_kldb isco_tool_vorlaufig isco_tool_mit.
##      <char>          <char>          <char>          <char>          <char>          <char>
## 1:      81883          81883          81883          81883          2433
## 2:      52112          52112          52112          52112          8322
## 3:      <NA>          <NA>          83131          83131          <NA>
```

```

## 4:          23411          23411    23411    23411          9329
## 5:          72134          72134    72132    72132          2412
## 6:          25104          25104    27224    27224          2144
##   res1_isco res2_isco kldb_infas_num          kldb_infas_str isco_infas_n
##   <char>    <char>    <char>          <char>          <cha
## 1:      2433      2433      81883          [81883] Pharmazie (s.s.T.) - Spezialist          24
## 2:      8322      8322      52112          [52112] Berufskraftfahrer(Pers./PKW.)-Fachkraft          83
## 3:      3412      3412      83131          [83131] Heilerziehungspflege, Sonderpaed. -Helfer          34
## 4:      7322      7322      23412          [23412] Drucktechnik - Fachkraft          73
## 5:      4312      4312      72132          [72132] Versicherungskaufleute - Fachkraft          43
## 6:      2149      2144      27224          [27224] Konstruktion und Geraetebau - Experte          21
##
##
## 1: [2433] Akademische und vergleichbare Fachkräfte im Bereich Vertrieb (Technik und Medizin, ohne In
## 2:                                     [8322] Personenkraftwagen-, T
## 3:                                     [3412] Nicht
## 4:
## 5:                                     [4312] Bürokräfte in der St
## 6:
##   kldb_tool kldb_human1 kldb_human2 isco_tool_1st res1_isco_1st res2_isco_1st item_id
##   <char>    <char>    <char>    <char>    <char>    <char>    <int>
## 1:         8         8         8         2         2         2         1
## 2:         5         5         5         8         8         8         2
## 3:        <NA>         8         8        <NA>         3         3         3
## 4:         2         2         2         9         7         7         4
## 5:         7         7         7         2         4         4         5
## 6:         2         2         2         2         2         2         6

```

```
#### 1) Select only KldB, as character
```

```

kldb_data <- data_basis %>%
  dplyr::select(item_id, kldb_human1, kldb_human2, kldb_tool) %>%
  mutate(across(c(kldb_human1, kldb_human2, kldb_tool), as.character))

```

```
### Remove all lines with NAs
```

```

kldb_data <- kldb_data %>%
  filter(!is.na(kldb_human1) & !is.na(kldb_human2) & !is.na(kldb_tool))

```

```
# Keep only two categories: "7" and "8"
```

```

kldb_data <- kldb_data %>%
  filter(kldb_human1 == "7" | kldb_human1 == "8") %>%
  filter(kldb_human2 == "7" | kldb_human2 == "8") %>%
  filter(kldb_tool == "7" | kldb_tool == "8")

```

```
# Reindex item_id (1, 2, 3,...)
```

```

kldb_data <- kldb_data %>%
  dplyr::mutate(item_id = dplyr::row_number())

```

```
# Collect all observed classes
```

```

kldb_levels <- kldb_data %>%
  dplyr::select(kldb_human1, kldb_human2, kldb_tool) %>%
  unlist(use.names = FALSE) %>%
  unique() %>%
  sort() %>%
  as.character() %>%

```

```

as.vector()

# K, number of distinct classes
K <- length(kldb_levels)
stopifnot(K >= 2)

# Helper: map original labels to 1..K using kldb_levels for stan compatibility
to_index <- function(v) as.integer(factor(v, levels = kldb_levels))

# 1) Map original labels to 1..K
kldb_data_idx <- kldb_data %>%
  dplyr::mutate(
    kldb_human1 = to_index(kldb_human1),
    kldb_human2 = to_index(kldb_human2),
    kldb_tool   = to_index(kldb_tool)
  )

# 2) Long format (now without NAs)
long_data <- kldb_data_idx %>%
  dplyr::transmute(item_id, r1 = kldb_human1, r2 = kldb_human2, r3 = kldb_tool) %>%
  tidyr::pivot_longer(cols = starts_with("r"), names_to = "rater", values_to = "y") %>%
  dplyr::mutate(rater_id = as.integer(factor(rater, levels = c("r1", "r2", "r3")))) %>%
  dplyr::arrange(item_id, rater_id)

# Robust 1..I indexing
item_ids <- sort(unique(long_data$item_id))
long_data$item_id_idx <- as.integer(factor(long_data$item_id, levels = item_ids))

I <- length(item_ids) # number of items (people interviewed)
J <- 3L # number of raters
N <- nrow(long_data) # number of observations (I * J if no missing)

cat("Items after filtering:", I, "\n")

## Items after filtering: 471
cat("Observations:", N, "\n")

## Observations: 1413
cat("Classes:", K, "\n")

## Classes: 2
cat("Class levels:", paste(kldb_levels, collapse = ", "), "\n")

## Class levels: 7, 8

# Packages
library(data.table)
library(dplyr)
library(tidyr)
library(cmdstanr)

```

```

# Write Stan model to file
stan_file <- "dawid_skene_kldb.stan"
writeLines(con = stan_file, text = '
  data {
    int<lower=2> K;           // number of categories
    int<lower=1> I;           // items
    int<lower=1> J;           // raters
    int<lower=1> N;           // number of observed ratings
    array[N] int<lower=1, upper=I> ii; // item index per observation
    array[N] int<lower=1, upper=J> jj; // rater index per observation
    array[N] int<lower=1, upper=K> y;  // observed category per observation

    vector<lower=0>[K] alpha;         // prior for pi
    array[K] vector<lower=0>[K] beta; // Dirichlet priors for theta rows, vary by true class
  }

  parameters {
    simplex[K] pi;                  // prevalence over true classes
    array[J, K] simplex[K] theta;   // confusion rows: for rater j and true class k
  }

  transformed parameters {
    // log_q_z[i, k] = log p(z_i = k | pi, theta) up to a constant
    array[I] vector[K] log_q_z;

    // Initialize with log pi
    for (i in 1:I) {
      log_q_z[i] = log(pi);
    }

    // Accumulate per observation
    for (n in 1:N) {
      int i = ii[n];
      int j = jj[n];
      int y_n = y[n];
      for (k in 1:K) {
        // Add log categorical pmf of y_n under theta[j, k, :]
        log_q_z[i, k] += categorical_lpmf(y_n | theta[j, k]);
      }
    }
  }

  model {
    // Priors
    pi ~ dirichlet(alpha);
    for (j in 1:J)
      for (k in 1:K)
        theta[j, k] ~ dirichlet(beta[k]);

    // Marginalized likelihood
    for (i in 1:I) {
      target += log_sum_exp(log_q_z[i]);
    }
  }

```

```

}

generated quantities {
  array[I] simplex[K] q_z;
  for (i in 1:I) {
    vector[K] lq = log_q_z[i];
    q_z[i] = softmax(lq);
  }
}
')

# Compile model once
mod <- cmdstan_model(stan_file, cpp_options = list(stan_threads = TRUE))

# Define 2 model configurations (alpha always uniform, only beta varies)
model_configs <- list(
  list(
    name = "alpha11_beta1",
    alpha = c(1.0, 1.0), # uniform prior
    beta = matrix(1.0, nrow = K, ncol = K) # uniform beta
  ),
  list(
    name = "alpha11_beta5diag",
    alpha = c(1.0, 1.0), # uniform prior
    beta = matrix(1.0, nrow = K, ncol = K) + diag(4.0, K) # 5 on diagonal, 1 elsewhere
  )
)

# Fit both models
fits <- list()
for (config in model_configs) {
  cat("\n=== Fitting model:", config$name, "===\n")

  stan_data <- list(
    I = I,
    J = J,
    K = K,
    N = N,
    ii = as.integer(long_data$item_id_idx),
    jj = as.integer(long_data$rater_id),
    y = as.integer(long_data$y),
    alpha = config$alpha,
    beta = config$beta
  )

  fit <- mod$sample(
    data = stan_data,
    seed = 1234,
    chains = 4,
    parallel_chains = 4,
    threads_per_chain = 1,
    iter_warmup = 1000,
    iter_sampling = 4000,

```

```

    refresh = 200,
    adapt_delta = 0.99
  )

fits[[config$name]] <- fit

# Check convergence
s <- fit$summary()
s$rhat <- as.numeric(s$rhat)
bad_rhat <- subset(s, rhat > 1.1 & grepl("^(\pi|\theta)\\b", variable))
if (nrow(bad_rhat) > 0) {
  cat("WARNING: R-hat values above 1.1:\n")
  print(bad_rhat[, c("variable", "rhat", "ess_bulk", "ess_tail")])
} else {
  cat("No R-hat values above 1.1\n")
}
}

```

```

##
## === Fitting model: alpha11_beta1 ===
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 4 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 1 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 4 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 2 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 1 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 3 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 2 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 1 Iteration:  1000 / 5000 [20%] (Warmup)
## Chain 1 Iteration:  1001 / 5000 [20%] (Sampling)
## Chain 3 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 4 Iteration:  1000 / 5000 [20%] (Warmup)
## Chain 4 Iteration:  1001 / 5000 [20%] (Sampling)
## Chain 2 Iteration:  1000 / 5000 [20%] (Warmup)
## Chain 2 Iteration:  1001 / 5000 [20%] (Sampling)
## Chain 1 Iteration:  1200 / 5000 [24%] (Sampling)
## Chain 3 Iteration:  1000 / 5000 [20%] (Warmup)
## Chain 3 Iteration:  1001 / 5000 [20%] (Sampling)
## Chain 2 Iteration:  1200 / 5000 [24%] (Sampling)
## Chain 4 Iteration:  1200 / 5000 [24%] (Sampling)
## Chain 1 Iteration:  1400 / 5000 [28%] (Sampling)

```

```

## Chain 3 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)

```

```

## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 55.4 seconds.
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 66.5 seconds.
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 68.0 seconds.
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 85.5 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 68.8 seconds.
## Total execution time: 85.6 seconds.
##
## No R-hat values above 1.1
##
## === Fitting model: alpha11_beta5diag ===
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 4 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 1 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 3 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 4 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 1 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 2 Iteration:  800 / 5000 [16%] (Warmup)
## Chain 3 Iteration:  800 / 5000 [16%] (Warmup)

```

```

## Chain 4 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 1 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 3 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 3 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 2 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 1 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 1 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 4 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 3 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 3 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)

```

```

## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 54.7 seconds.
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 59.5 seconds.
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 68.5 seconds.
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 80.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 65.9 seconds.
## Total execution time: 80.8 seconds.
##
## No R-hat values above 1.1

```

```

library(dplyr)
library(posterior)
library(ggplot2)
library(reshape2)

# Use kldb_levels (from data preparation chunk)
class_levels <- kldb_levels

```

```

# Function to analyze a single model
analyze_model <- function(fit, model_name, out_dir, class_levels, kldb_data) {
  dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

  save_plot <- function(p, filename, width = 7, height = 5, dpi = 600) {
    ggsave(
      filename = file.path(out_dir, filename),
      plot = p,
      width = width,
      height = height,
      dpi = dpi
    )
  }

  cat("\n=== Analyzing model:", model_name, "===\n")

  # -----
  # 0) Basic diagnostics for key parameters
  # -----
  diag_tbl <- fit$summary(variables = c("pi", "theta")) %>%
    dplyr::select(variable, rhat, ess_bulk, ess_tail)

  diag_bad <- diag_tbl %>%
    dplyr::filter(is.finite(rhat), rhat > 1.01) %>%
    dplyr::arrange(desc(rhat))

  if (nrow(diag_bad) > 0) {
    cat("Diagnostics with R-hat > 1.01:\n")
    print(diag_bad)
  }

  # -----
  # 1) Confusion matrices (theta), posterior mean and uncertainty
  # -----
  theta_draws_mat <- fit$draws("theta", format = "draws_matrix")
  theta_mean_vec <- colMeans(theta_draws_mat)

  # Expectation: theta has dimensions [J rater, K true, K given]
  theta_mean_arr <- array(
    theta_mean_vec,
    dim = c(J, K, K),
    dimnames = list(
      rater = c("human1", "human2", "tool"),
      true = class_levels,
      given = class_levels
    )
  )

  theta_human1_mean <- theta_mean_arr["human1", , ]
  theta_human2_mean <- theta_mean_arr["human2", , ]
  theta_tool_mean <- theta_mean_arr["tool", , ]

  plot_theta <- function(mat, title_txt) {

```

```

long <- reshape2::melt(mat, varnames = c("True", "Given"), value.name = "Prob")
ggplot(long, aes(x = Given, y = True, fill = Prob)) +
  geom_tile(color = "white") +
  scale_fill_gradient(low = "white", high = "darkgreen") +
  theme_light() +
  labs(
    title = title_txt,
    x = "Observed class",
    y = "True class C_i"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
}

p_theta_h1_mean <- plot_theta(theta_human1_mean, paste("Confusion matrix  $\pi^{(1)}$  (posterior mean) -", r
p_theta_h2_mean <- plot_theta(theta_human2_mean, paste("Confusion matrix  $\pi^{(2)}$  (posterior mean) -", r
p_theta_tool_mean <- plot_theta(theta_tool_mean, paste("Confusion matrix  $\pi^{(3)}$  (posterior mean) -", r

save_plot(p_theta_h1_mean, "theta_human1_mean.png")
save_plot(p_theta_h2_mean, "theta_human2_mean.png")
save_plot(p_theta_tool_mean, "theta_tool_mean.png")

# Posterior SD heatmaps for theta (uncertainty by cell)
theta_sd_vec <- apply(theta_draws_mat, 2, sd)
theta_sd_arr <- array(
  theta_sd_vec,
  dim = c(J, K, K),
  dimnames = list(
    rater = c("human1", "human2", "tool"),
    true = class_levels,
    given = class_levels
  )
)

p_theta_h1_sd <- plot_theta(theta_sd_arr["human1", , ], paste("Confusion matrix  $\pi^{(1)}$  (posterior SD)
p_theta_h2_sd <- plot_theta(theta_sd_arr["human2", , ], paste("Confusion matrix  $\pi^{(2)}$  (posterior SD)
p_theta_tool_sd <- plot_theta(theta_sd_arr["tool", , ], paste("Confusion matrix  $\pi^{(3)}$  (posterior SD)

save_plot(p_theta_h1_sd, "theta_human1_sd.png")
save_plot(p_theta_h2_sd, "theta_human2_sd.png")
save_plot(p_theta_tool_sd, "theta_tool_sd.png")

# -----
# 2) Class prevalence ( $\pi$ ), posterior mean and uncertainty
# -----
pi_means <- colMeans(as_draws_matrix(fit$draws("pi")))
pi_df <- data.frame(
  class = class_levels,
  prevalence = as.numeric(pi_means)
)

p_pi_bar <- ggplot(pi_df, aes(x = class, y = prevalence)) +
  geom_bar(stat = "identity", fill = "#02626f") +
  theme_light() +

```

```

labs(
  title = paste("Posterior mean of class prevalence (p) -", model_name),
  x = "Class",
  y = "Prevalence"
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

save_plot(p_pi_bar, "pi_mean_bar.png", width = 6, height = 4)

# -----
# 2b) Rater level scalar summaries from theta
# -----
pi_mean_vec <- as.numeric(pi_means)
names(pi_mean_vec) <- class_levels

weighted_diag_mass <- function(mat, w) sum(diag(mat) * w)
weighted_offdiag_mass <- function(mat, w) sum((1 - diag(mat)) * w)

rater_summary <- data.frame(
  rater = c("human1", "human2", "tool"),
  weighted_diag_mass = c(
    weighted_diag_mass(theta_human1_mean, pi_mean_vec),
    weighted_diag_mass(theta_human2_mean, pi_mean_vec),
    weighted_diag_mass(theta_tool_mean, pi_mean_vec)
  ),
  weighted_offdiag_mass = c(
    weighted_offdiag_mass(theta_human1_mean, pi_mean_vec),
    weighted_offdiag_mass(theta_human2_mean, pi_mean_vec),
    weighted_offdiag_mass(theta_tool_mean, pi_mean_vec)
  )
)

p_rater_diag <- ggplot(rater_summary, aes(x = rater, y = weighted_diag_mass)) +
  geom_bar(stat = "identity", fill = "#02626f") +
  theme_light() +
  labs(
    title = paste("Rater summary: Weighted diagonal mass -", model_name),
    x = "Rater",
    y = "Weighted diagonal mass"
  )

save_plot(p_rater_diag, "rater_weighted_diag_mass.png", width = 6, height = 4)

# -----
# 3) Posterior class probabilities per item ( $q_{i,j} = P(C_i = j \mid \text{data})$ )
# -----
qz_draws_mat <- fit$draws("q_z", format = "draws_matrix")
qz_mean_vec <- colMeans(qz_draws_mat)

qz_mean <- matrix(
  qz_mean_vec,
  nrow = I, ncol = K,
  dimnames = list(item = item_ids, class = class_levels)
)

```

```

)

qz_df <- as.data.frame(qz_mean)
p_cols <- paste0("p_", class_levels)
colnames(qz_df) <- p_cols
qz_df$item_id <- item_ids

# MAP label
qz_df$map_class_idx <- max.col(qz_df[p_cols], ties.method = "first")
qz_df$map_class_label <- class_levels[qz_df$map_class_idx]

# Uncertainty measures
eps <- 1e-12
qz_df$max_p <- apply(qz_df[p_cols], 1, max)

qz_df$entropy <- -rowSums(as.matrix(qz_df[p_cols]) * log(as.matrix(qz_df[p_cols]) + eps))

qz_df$margin <- apply(qz_df[p_cols], 1, function(p) {
  ps <- sort(p, decreasing = TRUE)
  ps[1] - ps[2]
})

# Credible set per item
credible_set <- function(p, classes, level = 0.95) {
  o <- order(p, decreasing = TRUE)
  cum <- cumsum(p[o])
  m <- which(cum >= level)[1]
  classes[o[seq_len(m)]]
}
qz_df$cs95 <- apply(qz_df[p_cols], 1, function(x) credible_set(x, class_levels, level = 0.95))

# Plots of uncertainty and confidence
p_maxp_hist <- ggplot(qz_df, aes(x = max_p)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior max probability", y = "Count",
       title = paste("Model confidence per item -", model_name))

p_entropy_hist <- ggplot(qz_df, aes(x = entropy)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior entropy", y = "Count",
       title = paste("Item uncertainty (entropy) -", model_name))

p_margin_hist <- ggplot(qz_df, aes(x = margin)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior margin (top1 minus top2)", y = "Count",
       title = paste("Item uncertainty (margin) -", model_name))

save_plot(p_maxp_hist, "qz_maxp_hist.png", width = 6, height = 4)
save_plot(p_entropy_hist, "qz_entropy_hist.png", width = 6, height = 4)
save_plot(p_margin_hist, "qz_margin_hist.png", width = 6, height = 4)

```

```

# -----
# 3b) MAP distribution vs pi (sanity check)
# -----
map_tab <- qz_df %>%
  count(map_class_label, name = "n_map") %>%
  mutate(map_share = n_map / sum(n_map)) %>%
  rename(class = map_class_label)

pi_tab <- data.frame(class = class_levels, pi_mean = as.numeric(pi_means))

map_vs_pi <- left_join(map_tab, pi_tab, by = "class")

p_map_vs_pi <- ggplot(map_vs_pi, aes(x = class)) +
  geom_col(aes(y = map_share), fill = "#02626f") +
  geom_point(aes(y = pi_mean), size = 2) +
  theme_light() +
  labs(
    title = paste("MAP share (bars) vs posterior mean p (points) -", model_name),
    x = "Class",
    y = "Share / prevalence"
  )

save_plot(p_map_vs_pi, "map_share_vs_pi.png", width = 6, height = 4)

# Return summary statistics
list(
  model_name = model_name,
  pi_means = pi_means,
  pi_df = pi_df,
  theta_human1_mean = theta_human1_mean,
  theta_human2_mean = theta_human2_mean,
  theta_tool_mean = theta_tool_mean,
  rater_summary = rater_summary,
  qz_df = qz_df,
  map_vs_pi = map_vs_pi
)
}

# Analyze all models and store results
results_list <- list()
for (model_name in names(fits)) {
  out_dir <- file.path("img/model_results/kldb/beta_comparison", model_name)
  results_list[[model_name]] <- analyze_model(
    fits[[model_name]],
    model_name,
    out_dir,
    class_levels,
    kldb_data
  )
}

##
## === Analyzing model: alpha11_beta1 ===

```

```

##
## === Analyzing model: alpha11_beta5diag ===
library(dplyr)
library(ggplot2)
library(reshape2)
library(gridExtra)

# Create comparison directory
comp_dir <- "img/model_results/kldb/beta_comparison"
dir.create(comp_dir, recursive = TRUE, showWarnings = FALSE)

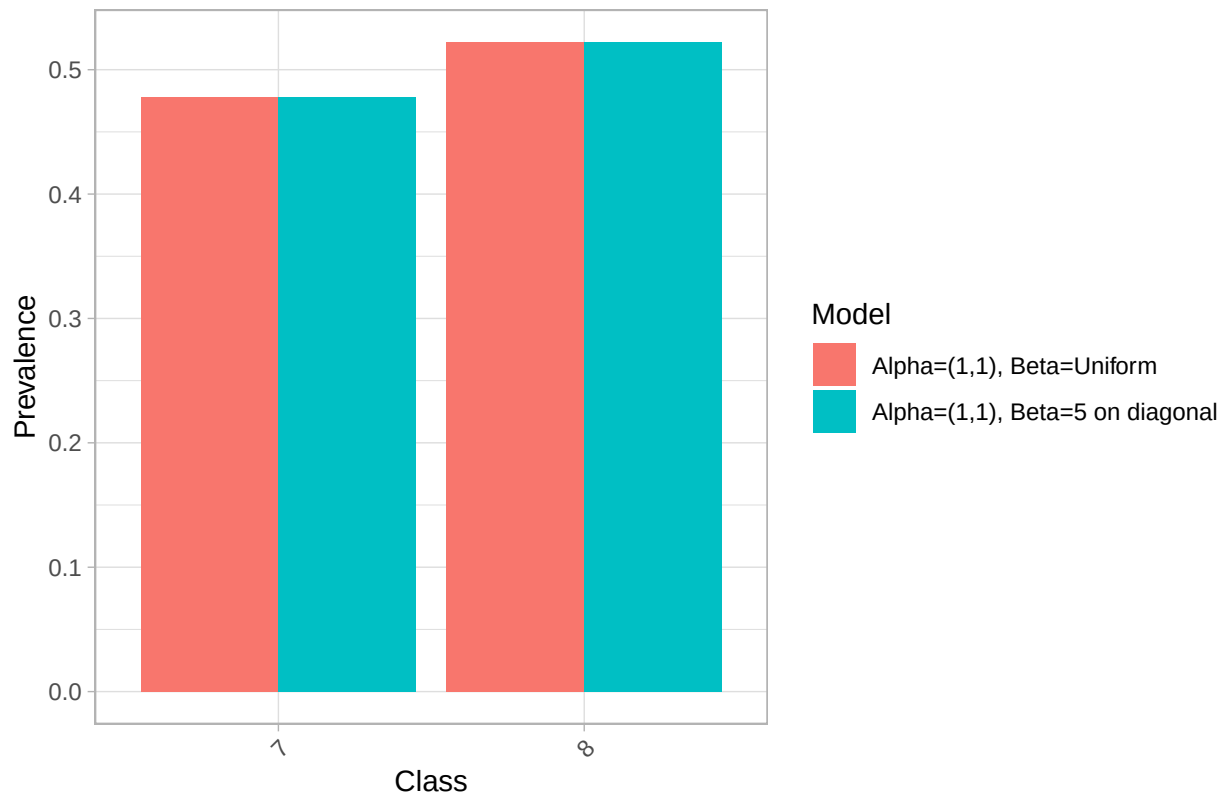
# -----
# 1) Compare pi (prevalence) across models
# -----
pi_comparison <- data.frame()
for (model_name in names(results_list)) {
  pi_df <- results_list[[model_name]]$pi_df
  pi_df$model <- model_name
  pi_comparison <- rbind(pi_comparison, pi_df)
}

p_pi_comparison <- ggplot(pi_comparison, aes(x = class, y = prevalence, fill = model)) +
  geom_bar(stat = "identity", position = "dodge") +
  scale_fill_discrete(
    labels = c(
      "alpha11_beta1" = "Alpha=(1,1), Beta=Uniform",
      "alpha11_beta5diag" = "Alpha=(1,1), Beta=5 on diagonal"
    )
  ) +
  theme_light() +
  labs(
    title = "Comparison of posterior mean p across models (KldB categories 7 and 8)",
    x = "Class",
    y = "Prevalence",
    fill = "Model"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

ggsave(file.path(comp_dir, "pi_comparison.png"), p_pi_comparison, width = 8, height = 5, dpi = 600)
p_pi_comparison

```

Comparison of posterior mean p across models (KldB categories 7 and 8)



```
# -----
# 2) Compare theta matrices (confusion matrices) across models
# -----
# Extract diagonal elements (accuracy) for each rater
theta_diag_comparison <- data.frame()
for (model_name in names(results_list)) {
  theta_h1 <- results_list[[model_name]]$theta_human1_mean
  theta_h2 <- results_list[[model_name]]$theta_human2_mean
  theta_tool <- results_list[[model_name]]$theta_tool_mean

  for (k in 1:K) {
    theta_diag_comparison <- rbind(
      theta_diag_comparison,
      data.frame(
        model = model_name,
        rater = "human1",
        true_class = class_levels[k],
        accuracy = theta_h1[k, k]
      ),
      data.frame(
        model = model_name,
        rater = "human2",
        true_class = class_levels[k],
        accuracy = theta_h2[k, k]
      ),
      data.frame(
```

```

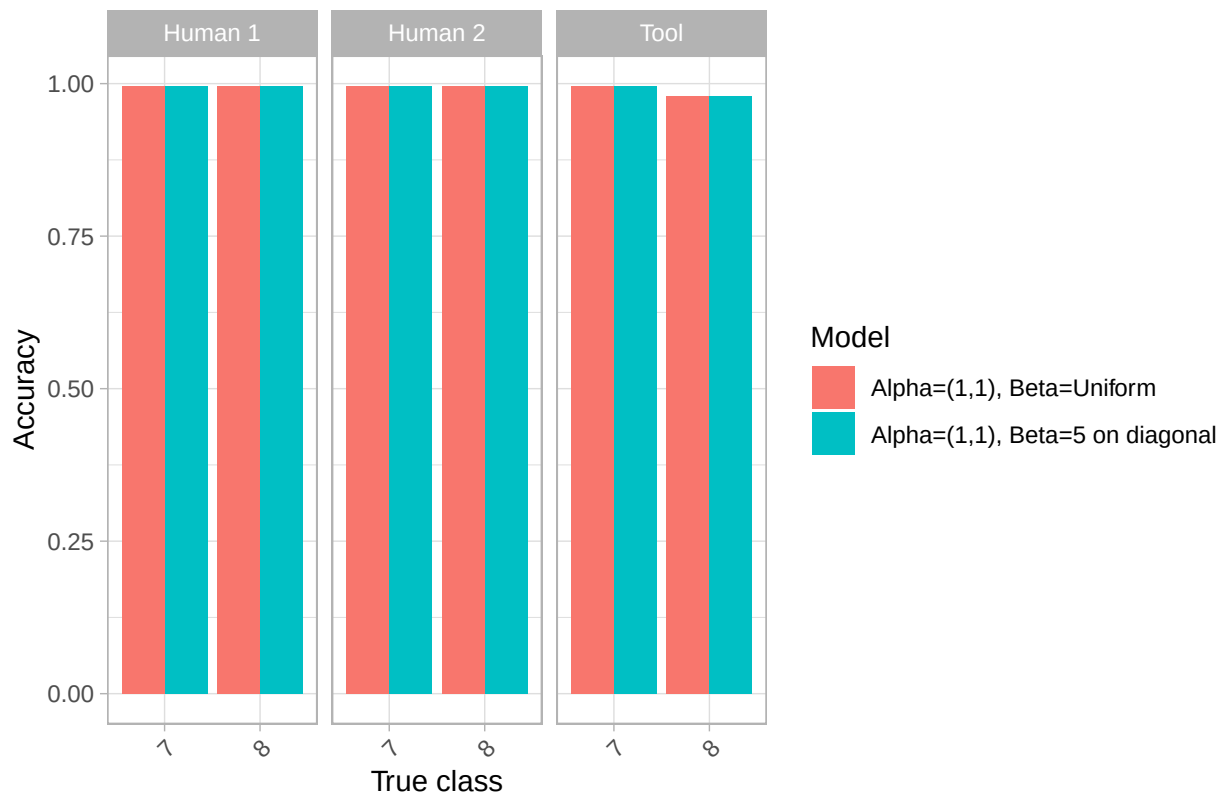
        model = model_name,
        rater = "tool",
        true_class = class_levels[k],
        accuracy = theta_tool[k, k]
    )
}
}

p_theta_diag_comparison <- ggplot(theta_diag_comparison,
                                aes(x = true_class, y = accuracy, fill = model)) +
  geom_bar(stat = "identity", position = "dodge") +
  scale_fill_discrete(
    labels = c(
      "alpha11_beta1" = "Alpha=(1,1), Beta=Uniform",
      "alpha11_beta5diag" = "Alpha=(1,1), Beta=5 on diagonal"
    )
  ) +
  facet_wrap(~ rater, ncol = 3,
             labeller = labeller(rater = c("human1" = "Human 1", "human2" = "Human 2", "tool" = "Tool")))
  theme_light() +
  labs(
    title = "Comparison of diagonal elements (accuracy) across models",
    x = "True class",
    y = "Accuracy",
    fill = "Model"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

ggsave(file.path(comp_dir, "theta_diag_comparison.png"), p_theta_diag_comparison,
        width = 12, height = 5, dpi = 600)
p_theta_diag_comparison

```

Comparison of diagonal elements (accuracy) across models



```
# -----
# 3) Compare weighted diagonal mass (rater summary)
# -----
rater_summary_comparison <- data.frame()
for (model_name in names(results_list)) {
  rater_sum <- results_list[[model_name]]$rater_summary
  rater_sum$model <- model_name
  rater_summary_comparison <- rbind(rater_summary_comparison, rater_sum)
}

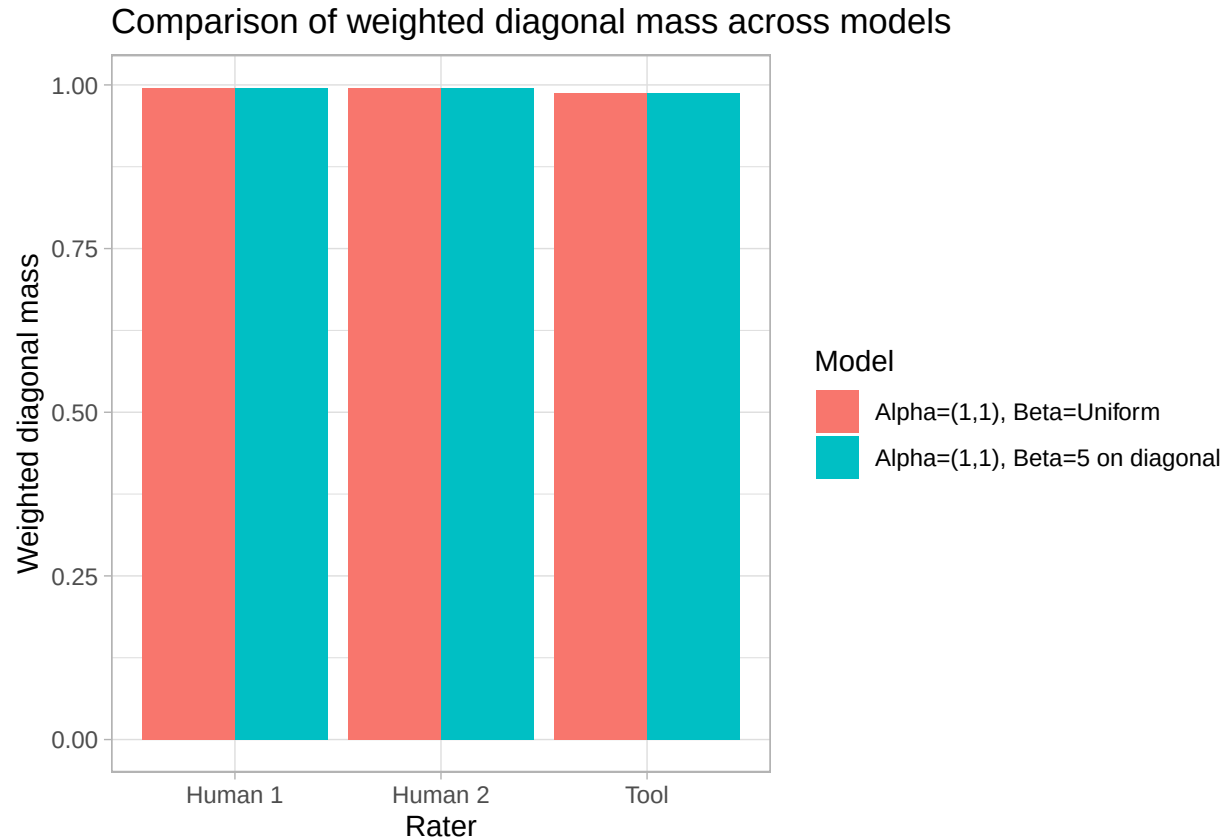
p_rater_diag_comparison <- ggplot(rater_summary_comparison,
  aes(x = rater, y = weighted_diag_mass, fill = model)) +
  geom_bar(stat = "identity", position = "dodge") +
  scale_fill_discrete(
    labels = c(
      "alpha11_beta1" = "Alpha=(1,1), Beta=Uniform",
      "alpha11_beta5diag" = "Alpha=(1,1), Beta=5 on diagonal"
    )
  ) +
  scale_x_discrete(
    labels = c("human1" = "Human 1", "human2" = "Human 2", "tool" = "Tool")
  ) +
  theme_light() +
  labs(
    title = "Comparison of weighted diagonal mass across models",
    x = "Rater",
```

```

    y = "Weighted diagonal mass",
    fill = "Model"
  )

ggsave(file.path(comp_dir, "rater_diag_mass_comparison.png"), p_rater_diag_comparison,
        width = 8, height = 5, dpi = 600)
p_rater_diag_comparison

```



```

# -----
# 4) Compare item-level uncertainty (entropy) distributions
# -----

entropy_comparison <- data.frame()
for (model_name in names(results_list)) {
  qz_df <- results_list[[model_name]]$qz_df
  entropy_comparison <- rbind(
    entropy_comparison,
    data.frame(
      model = model_name,
      entropy = qz_df$entropy
    )
  )
}

p_entropy_comparison <- ggplot(entropy_comparison, aes(x = entropy, fill = model)) +
  geom_density(alpha = 0.5) +
  scale_fill_discrete(

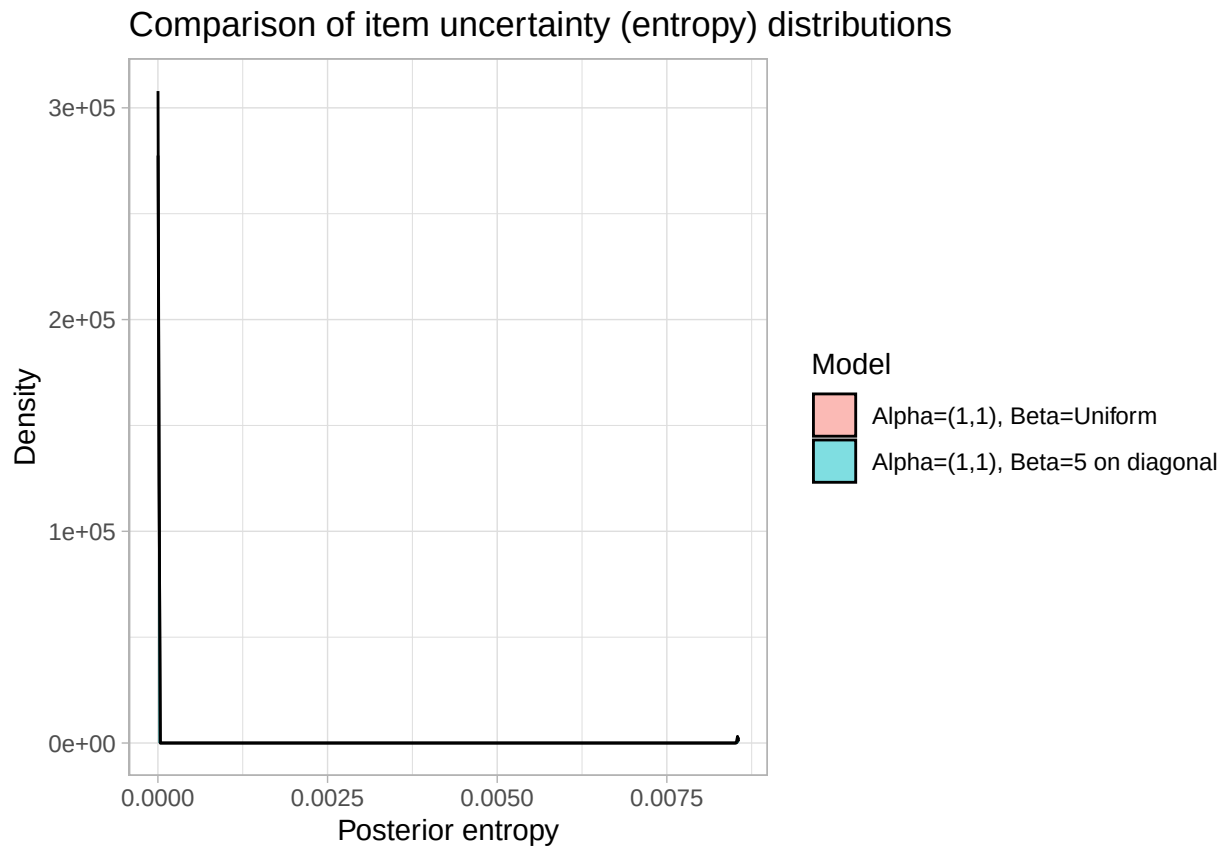
```

```

labels = c(
  "alpha11_beta1" = "Alpha=(1,1), Beta=Uniform",
  "alpha11_beta5diag" = "Alpha=(1,1), Beta=5 on diagonal"
)
) +
theme_light() +
labs(
  title = "Comparison of item uncertainty (entropy) distributions",
  x = "Posterior entropy",
  y = "Density",
  fill = "Model"
)

ggsave(file.path(comp_dir, "entropy_comparison.png"), p_entropy_comparison,
        width = 8, height = 5, dpi = 600)
p_entropy_comparison

```



```

# Boxplot version
p_entropy_box <- ggplot(entropy_comparison, aes(x = model, y = entropy, fill = model)) +
  geom_boxplot() +
  scale_fill_discrete(
    labels = c(
      "alpha11_beta1" = "Alpha=(1,1), Beta=Uniform",
      "alpha11_beta5diag" = "Alpha=(1,1), Beta=5 on diagonal"
    )
  ) +

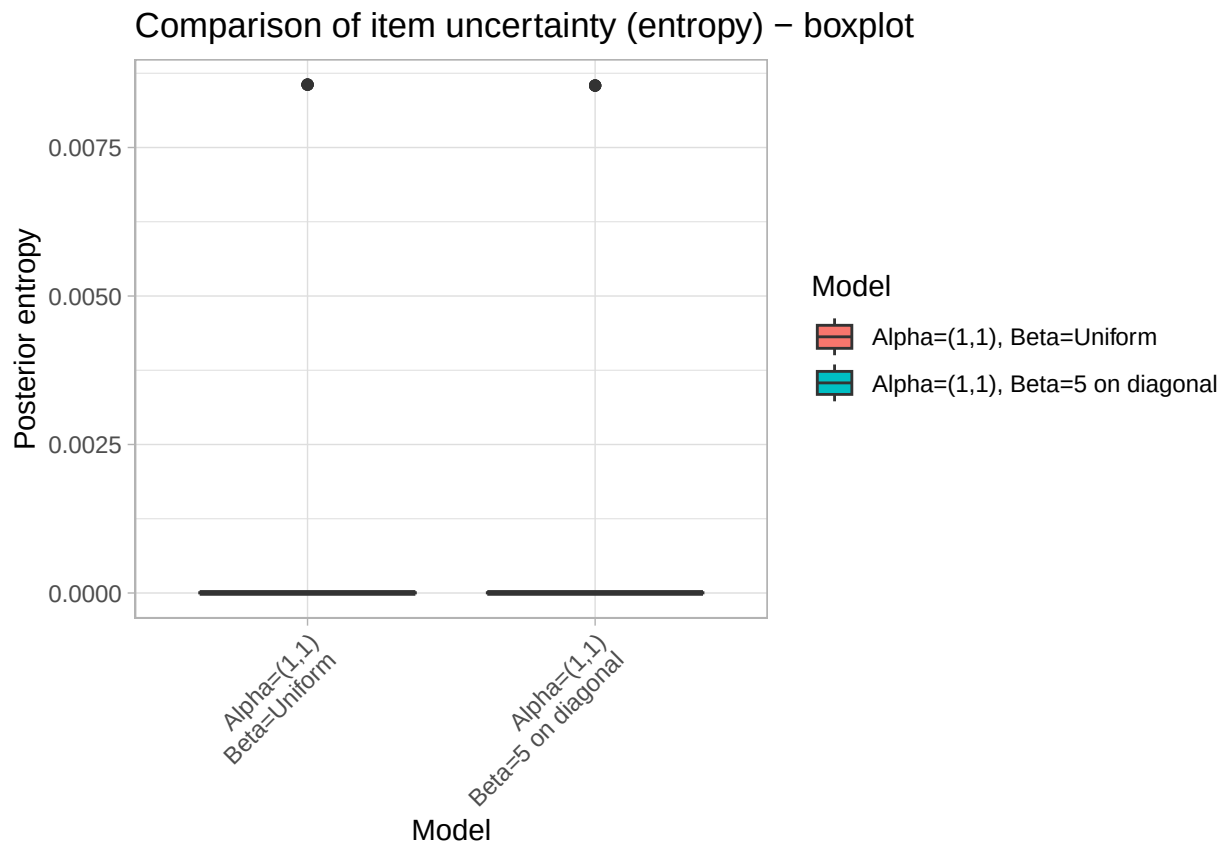
```

```

scale_x_discrete(
  labels = c(
    "alpha11_beta1" = "Alpha=(1,1)\nBeta=Uniform",
    "alpha11_beta5diag" = "Alpha=(1,1)\nBeta=5 on diagonal"
  )
) +
theme_light() +
labs(
  title = "Comparison of item uncertainty (entropy) - boxplot",
  x = "Model",
  y = "Posterior entropy",
  fill = "Model"
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

ggsave(file.path(comp_dir, "entropy_boxplot.png"), p_entropy_box,
  width = 8, height = 5, dpi = 600)
p_entropy_box

```



```

# -----
# 5) Summary table of key statistics
# -----
summary_table <- data.frame()
for (model_name in names(results_list)) {
  pi_means <- results_list[[model_name]]$pi_means
  rater_sum <- results_list[[model_name]]$rater_summary
}

```

```

qz_df <- results_list[[model_name]]$qz_df

summary_table <- rbind(
  summary_table,
  data.frame(
    model = model_name,
    pi_class7 = pi_means[1],
    pi_class8 = pi_means[2],
    h1_weighted_diag = rater_sum$weighted_diag_mass[rater_sum$rater == "human1"],
    h2_weighted_diag = rater_sum$weighted_diag_mass[rater_sum$rater == "human2"],
    tool_weighted_diag = rater_sum$weighted_diag_mass[rater_sum$rater == "tool"],
    mean_entropy = mean(qz_df$entropy),
    median_entropy = median(qz_df$entropy),
    mean_max_p = mean(qz_df$max_p),
    median_max_p = median(qz_df$max_p)
  )
)
}

print(summary_table)

##              model pi_class7 pi_class8 h1_weighted_diag h2_weighted_diag tool_weighted_diag me
## pi[1]          alpha11_beta1 0.4781230 0.5218770          0.9957709          0.9957859          0.9874123 7.
## pi[1]1 alpha11_beta5diag 0.4778968 0.5221032          0.9958726          0.9958791          0.9875967 7.
##          median_entropy mean_max_p median_max_p
## pi[1]          1.392604e-06 0.9999905          0.9999999
## pi[1]1          1.315620e-06 0.9999905          0.9999999

# Save summary table
write.csv(summary_table, file.path(comp_dir, "summary_comparison.csv"), row.names = FALSE)

# -----
# 6) Detailed theta comparison - heatmaps side by side
# -----
# Create a function to plot all theta matrices for comparison
plot_theta_comparison <- function(rater_name, rater_idx) {
  plots_list <- list()
  for (model_name in names(results_list)) {
    if (rater_name == "human1") {
      theta_mat <- results_list[[model_name]]$theta_human1_mean
    } else if (rater_name == "human2") {
      theta_mat <- results_list[[model_name]]$theta_human2_mean
    } else {
      theta_mat <- results_list[[model_name]]$theta_tool_mean
    }
  }

  long <- reshape2::melt(theta_mat, varnames = c("True", "Given"), value.name = "Prob")
  rater_label <- switch(rater_name,
    "human1" = "Human 1",
    "human2" = "Human 2",
    "tool" = "Tool",
    rater_name)
  p <- ggplot(long, aes(x = Given, y = True, fill = Prob)) +
    geom_tile(color = "white") +

```

```

scale_fill_gradient(low = "white", high = "darkgreen", limits = c(0, 1)) +
theme_light() +
labs(
  title = paste0(model_name, "\n", rater_label),
  x = "Observed class",
  y = "True class C_i"
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

plots_list[[model_name]] <- p
}

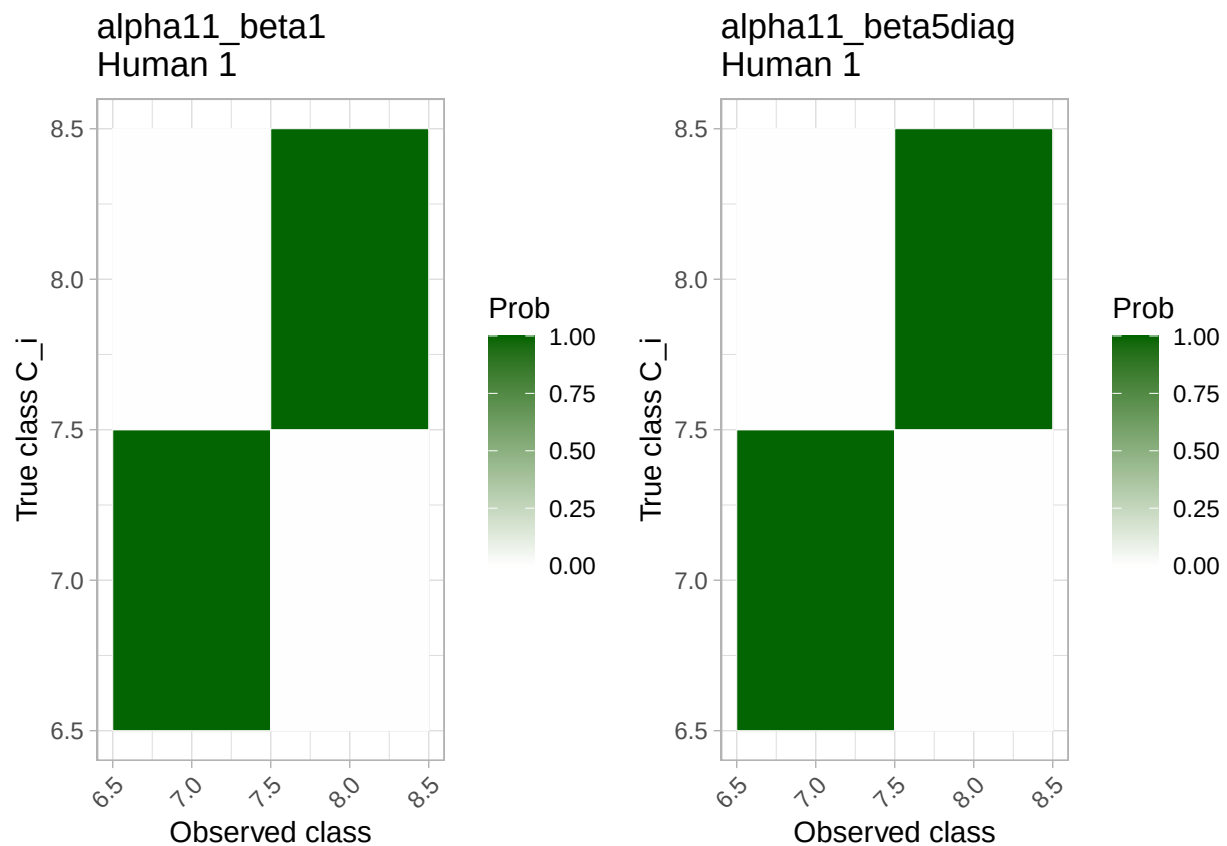
# Combine plots (2 models side by side)
p_combined <- do.call(grid.arrange, c(plots_list, ncol = 2))

ggsave(file.path(comp_dir, paste0("theta_comparison_", rater_name, ".png")),
  p_combined, width = 14, height = 6, dpi = 600)

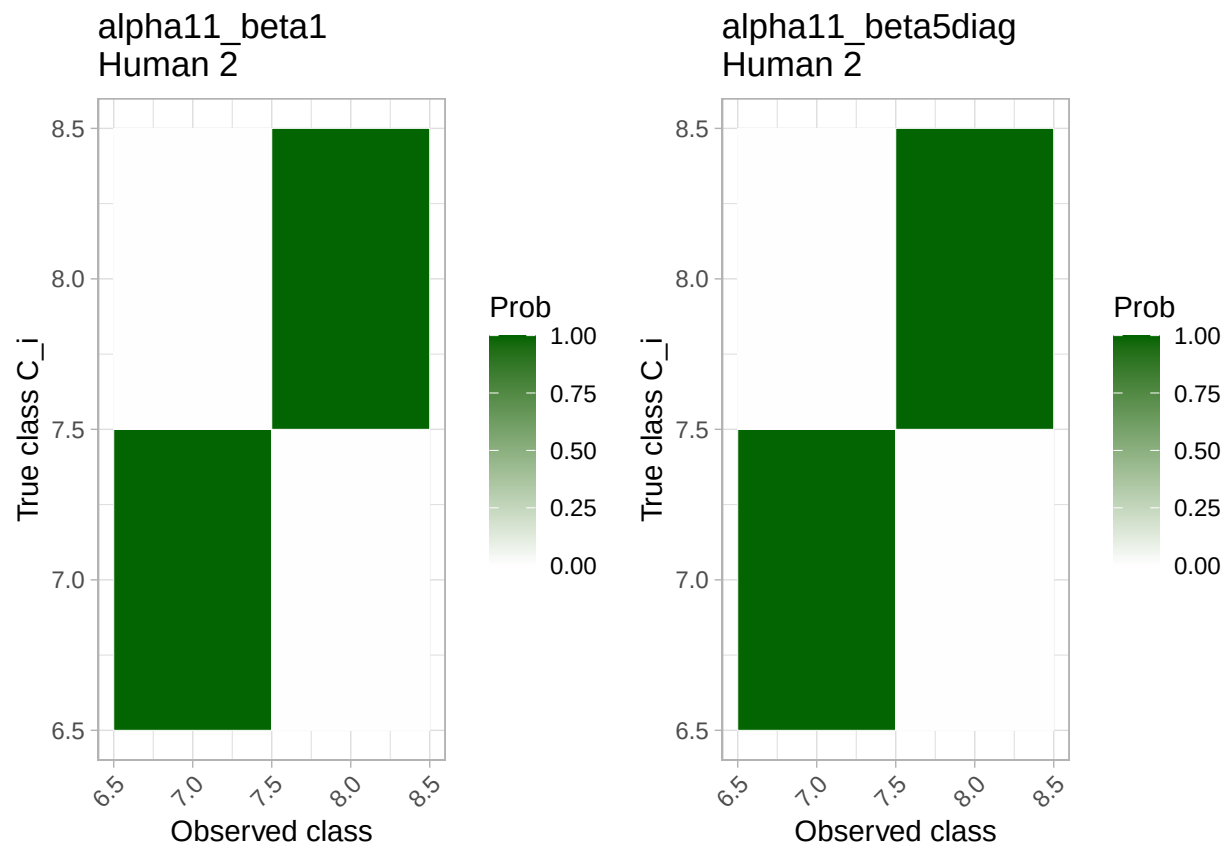
return(p_combined)
}

# Create comparison plots for each rater
p_theta_h1_comp <- plot_theta_comparison("human1", 1)

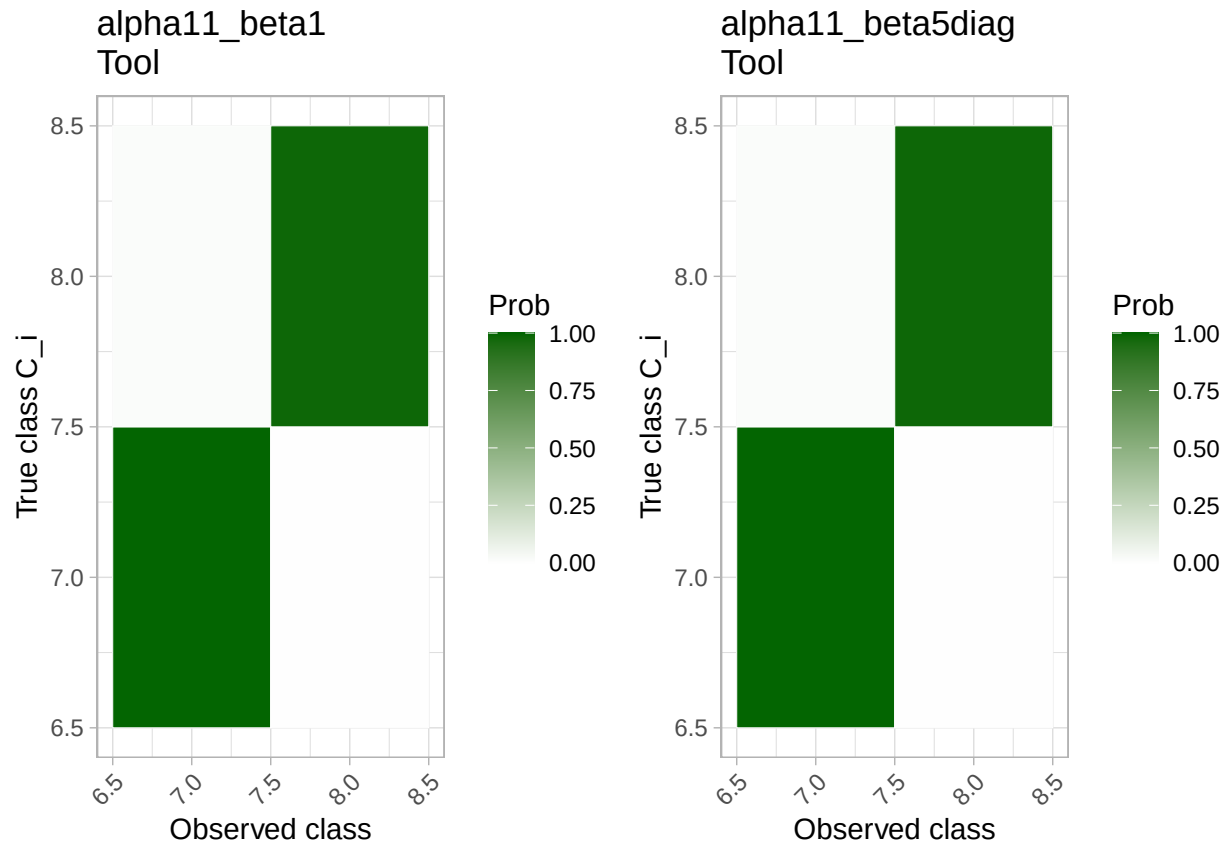
```



```
p_theta_h2_comp <- plot_theta_comparison("human2", 2)
```



```
p_theta_tool_comp <- plot_theta_comparison("tool", 3)
```



```
cat("\n=== Comparison complete ===\n")
```

```
##
```

```
## === Comparison complete ===
```

```
cat("All comparison plots saved to:", comp_dir, "\n")
```

```
## All comparison plots saved to: img/model_results/kldb/beta_comparison
```

```
cat("Models compared:\n")
```

```
## Models compared:
```

```
cat(" - alpha11_beta1: alpha = (1,1), beta = uniform\n")
```

```
## - alpha11_beta1: alpha = (1,1), beta = uniform
```

```
cat(" - alpha11_beta5diag: alpha = (1,1), beta = 5 on diagonal\n")
```

```
## - alpha11_beta5diag: alpha = (1,1), beta = 5 on diagonal
```